Knowledge Intelligence System

Project Draft

1. Vision & Overview

The goal of this project is to build a **Retrieval-Augmented Generation (RAG)**-based **Knowledge Intelligence System** that allows users to ingest, organize, search, and converse with their internal documents and data sources using a conversational AI interface. The system will combine a **vector-based retrieval layer** with a **large language model (LLM)** to provide accurate, context-aware answers grounded in the user's knowledge base, while offering an admin-friendly interface for managing content and monitoring usage.

2. Primary Objectives

- **Centralized knowledge hub:** Allow users to upload and manage diverse document types (PDF, DOCX, text, web pages, etc.) in one place.
- **High-quality retrieval:** Use embeddings and vector search to find the most relevant chunks of information for each query.
- **LLM-based reasoning:** Use an LLM to synthesize retrieved context and generate coherent, helpful answers.
- **Traceability & citations:** Show which documents and passages were used to answer a query.
- **User-friendly interface:** Provide an intuitive web UI for end-users and admins.
- **Security & access control (MVP-lite):** Basic separation of workspaces/collections and API keys; avoid storing user secrets in logs.
- **Extensibility:** Architect the system so data sources, models, and retrieval strategies can be swapped or extended later.

3. Target Users & Use Cases

- **Internal teams (knowledge workers / researchers):**
  - Ask natural language questions over their reports, research notes, policy docs, etc.
  - Quickly find supporting evidence, quotes, and summaries.
- **Customer support / documentation teams:**
  - Use conversational search over FAQs, manuals, and support docs.
- **Individual professionals / students:**
  - Upload personal notes and articles and query them as a personalized knowledge assistant.

Initial release will target a **single-tenant deployment** (one organization or one user per deployment) to keep security scope manageable.

4. Scope for Level 3 App (MVP)

4.1 In-Scope Features

- **Document ingestion**
  - Upload documents via web UI (start with PDF, plain text, and Markdown).
  - Basic parsing and chunking pipeline with configurable chunk size/overlap.
  - Metadata extraction (title, filename, upload date, tags).
- **Embedding & storage**
  - Generate embeddings for chunks using a chosen embedding model.
  - Store chunks + metadata + embeddings in a vector store (e.g., local DB or cloud vector DB).
  - Simple collection/workspace concept to group related documents.
- **Query & retrieval**
  - Text query box in the UI.
  - Retrieve top-k relevant chunks from selected collection(s).
  - Option to adjust top-k and similarity threshold via settings.
- **LLM question-answering**

        - Build RAG prompt with retrieved chunks and user question.
        - Generate answer via LLM, with:
            - Inline citations (chunk/document references).
            - Optional "show sources" section listing documents used.
- **Conversation interface**
    - Persistent chat-like interface per collection/workspace.
    - Show both questions and responses, along with source snippets.
- **Administration**
    - View list of documents with metadata and status (indexed/not indexed).
    - Re-index or delete documents.
    - Configuration page for:
        - Model/API keys (stored securely, e.g., in env variables).
        - Chunking parameters and retrieval settings.
- **Logging & monitoring (basic)**
    - Store query logs with timestamps, selected collection, and references to used chunks.
    - Simple analytics page (e.g., recent queries, top documents used).

4.2 Out-of-Scope (for MVP)

- Advanced multi-tenant RBAC and SSO.
- Real-time collaborative editing of documents.
- Complex workflows (approval flows, document versioning).
- Fine-tuning or training custom models.
- Mobile apps (use responsive web design instead).

These can be considered for later phases.

5. High-Level Architecture

- **Frontend (Web App)**
    - Built with a modern framework (e.g., React/Next.js or similar).
    - Pages:
        - Login / API key setup (if relevant).
        - Dashboard (collections, recent chats).
        - Collection detail (documents, chat, settings).
        - Admin/configuration panel.
    - Communicates with backend via REST/GraphQL API.
- **Backend (API + Orchestration)**
    - Ingests documents, manages chunking and embedding.
    - Manages collections/workspaces and metadata.
    - Handles query flow:
        - Accept question -> retrieve chunks -> build prompt -> call LLM -> return answer + sources.
    - Provides endpoints for:
        - Document upload & management.
        - Chat / RAG query.
        - Configuration & analytics.
- **Embedding & Vector Store Layer**
    - Embedding service (could be external API or local model).
    - Vector store (e.g., a local DB with vector extension, or a dedicated vector DB).
    - Abstraction layer to later support swapping providers.
- **LLM Layer**
    - Abstraction over different LLM providers (e.g., OpenAI, local LLM).
    - RAG pipeline: prompt templates, context injection, answer generation.
- **Storage**
    - Document storage (filesystem or object storage).
    - Relational/NoSQL DB for metadata and logs.

    - Vector store for embeddings (could be same DB or separate).

6. Data Flow (End-to-End)

1. **Document upload**
    - User uploads document via UI.
    - Backend stores raw file and enqueues ingestion job (if using a queue) or processes synchronously.
    - Document is parsed into text, then chunked.
    - Chunks are embedded and saved in the vector store with metadata.
2. **User query**
    - User selects collection and submits a question.
    - Backend generates query embedding and retrieves top-k chunks from that collection.
    - Backend constructs a prompt combining:
        - System instructions (grounding, style).
        - User question.
        - Retrieved context chunks (with source identifiers).
    - LLM generates an answer.
    - Backend returns:
        - Answer text.
        - List of source snippets (for citations).
        - Any metadata needed for logging/analytics.
    - Frontend displays answer, with clickable citations to open source excerpts.
3. **Feedback & logging**
    - Each interaction is stored in the DB for history and analytics.
    - (Optional later) Users can thumbs-up/down answers to improve heuristics.

7. Key Design Decisions & Trade-Offs

- **Chunking strategy**
    - Start with fixed-size token/character-based chunks with overlaps.
    - Later, explore semantic or section-based chunking for better coherence.
- **Vector store choice**
    - For Level 3, prioritize **ease of setup and local development**.
    - Start with a vector DB that has a simple SDK and works locally.
- **Model provider**
    - For this project, assume availability of a hosted LLM and embedding API.
    - Keep model access abstracted behind a service interface for easy swapping.
- **Context window & prompt size**
    - Limit the number of chunks and their sizes to avoid exceeding model context window.
    - Implement a simple context budget mechanism: sort by relevance and trim to fit.
- **Security**
    - Use API keys and environment variables for external services.
    - Ensure documents and embeddings are stored locally or in a controlled environment.
    - Avoid logging raw documents or PII in plain text logs.

8. Non-Functional Requirements

- **Performance**
    - Typical question-answer round trip: **< 5–7 seconds** for medium-sized corpora.
    - Incremental ingestion: new documents should become searchable within acceptable time (e.g., < 1–2 minutes depending on size).
- **Scalability**
    - Design for a **few hundred thousand chunks** in MVP.
    - Avoid hard-coding limits that prevent later scaling (e.g., keep batch operations configurable).
- **Reliability**
    - Handle transient model/vector store errors gracefully with retries.

    - Ensure ingestion jobs are idempotent (re-running does not duplicate data).
- **Usability**
    - Simple, guided flows for:
        - Creating a collection.
        - Uploading documents.
        - Asking questions.
    - Clear error messages and loading states in the UI.

9. Initial Technology Stack (Proposed)

- **Frontend**
    - React with a component library (e.g., Tailwind + Headless UI, or Material UI).
- **Backend**
    - Node.js / TypeScript backend (e.g., Express, Fastify, or Next.js API routes) *or* Python (FastAPI) — pick one and keep interfaces clear.
- **Storage**
    - Relational DB (e.g., PostgreSQL or SQLite for local dev) for metadata, logs, and configuration.
    - Vector store (e.g., pgvector extension, or a dedicated vector DB service).
- **Infra / DevOps**
    - Environment configuration via .env files.
    - Containerization with Docker (optional but recommended).
    - Basic logging using app-level logger (structured logs where possible).

10. Milestones & Deliverables

- **Milestone 1 — Project Setup & Skeleton**
    - Initialize repo structure for frontend and backend.
    - Set up base pages/routes and API health check.
    - Configure environment variables and basic config management.
- **Milestone 2 — Document Ingestion Pipeline**
    - Implement document upload endpoint and UI.
    - Implement parsing and chunking for PDFs and text/Markdown.
    - Implement embedding generation and vector store integration.
    - Basic collection/workspace management.
- **Milestone 3 — RAG Query Flow**
    - Implement query endpoint: embed query, retrieve context, call LLM.
    - Implement prompt templates for Q&A with citations.
    - Build chat UI with history per collection.
- **Milestone 4 — Admin & Analytics**
    - Document list view with status and actions (re-index, delete).
    - Query log storage and simple analytics view (recent queries, top documents).
    - Basic configuration UI for retrieval parameters.
- **Milestone 5 — Hardening & UX Polish**
    - Error handling, loading states, and validation.
    - Performance tuning of ingestion and query flow.
    - Documentation: README, architecture overview, and setup guide.

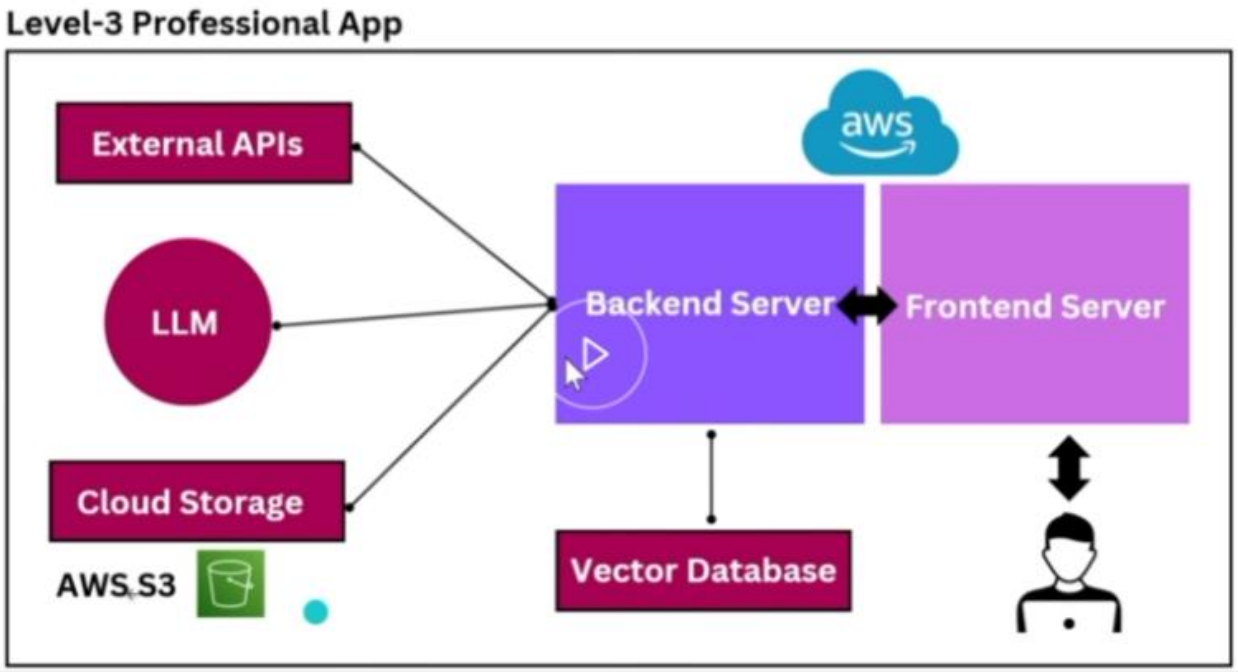
11. Risks & Mitigations

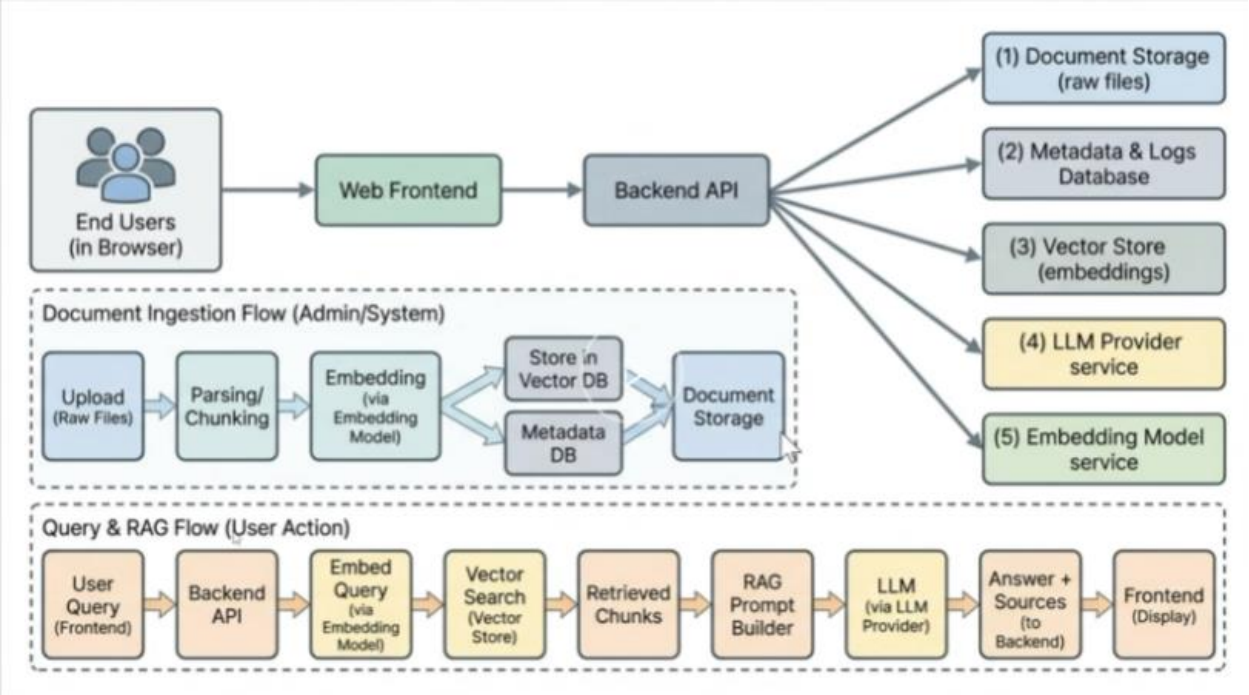
- **Model or embedding API limits**
    - Mitigation: Batch embedding requests, cache results, and implement backoff/retry.
- **Hallucinations / incorrect answers**
    - Mitigation: Strong grounding instructions in prompts, aggressive use of citations, explicit indication when answer is uncertain or out-of-scope.
- **Large documents / corpora**
    - Mitigation: Streaming ingestion, configurable chunking, and index monitoring.
- **User adoption (complex UI)**

- Mitigation: Keep flows minimal and intuitive; iterate based on user feedback.
-

12. Next Steps

- Finalize tech stack choice (Node/TS vs Python, specific vector store).
- Define concrete data models (collections, documents, chunks, messages, users).
- Create detailed API design (endpoints and contracts).





PROJECT BRIEF

MEMORANDUM
TO: Lead Data & ML Engineer
FROM: Engineering Supervisor
DATE: February 25, 2026
SUBJECT: Project Brief: Knowledge Intelligence System (RAG MVP) & Deployment Pipeline

This document serves as the formal project brief for the Level 3 MVP of our internal Knowledge Intelligence System. You are tasked with leading the end-to-end technical execution of this Retrieval-Augmented Generation (RAG) system, taking it from core API development through to full containerization and CI/CD automation.

This must be treated as an enterprise-grade application from day one. Expect strict adherence to MLOps and LLMOps best practices.

---

1. Project Overview & Scope
We are building a centralized, conversational RAG system allowing internal teams to query diverse document types (PDF, text, Markdown) securely.
In-Scope (MVP):
- Document ingestion, parsing, chunking, and metadata extraction.
- Vector embedding generation and semantic retrieval.
- LLM orchestration for Q&A with strict inline citations.
- Collection/Workspace isolation (Single-tenant).
- Admin dashboard for document management and basic query logging.

Out-of-Scope:
- Multi-tenant RBAC, fine-tuning, real-time collaboration.

2. Architecture & Tech Stack Definition
To align with our AI infrastructure, we are standardizing the stack around Python for the backend to ensure seamless integration with the embedding models and LLM orchestrators.
- **Backend / API:** Python (FastAPI).
- **Frontend:** React / Next.js (Tailwind).
- **Database (Relational):** PostgreSQL (for metadata, query logs, configuration).
- **Database (Vector):** pgvector (via Postgres) or a dedicated local vector DB container (e.g., Qdrant/Milvus).
- **LLM/Embeddings:** Abstracted interface (start with a hosted LLM, but build the embedding pipeline to run locally for testing).
- **Configuration Management (CM):** Docker & Docker Compose.
- **CI/CD:** GitHub Actions / GitLab CI.

3. Execution Phases & Deliverables
Phase 1: Configuration Management (CM) & Local Environment
Given recent development cycles, prioritize robust, clean Dockerfile configurations from day one.
- **Deliverable:** Containerized skeleton for both the FastAPI backend and Next.js frontend.
- **Directive:** Set up docker-compose.yml to spin up the APIs, PostgreSQL, and the Vector Store simultaneously.
- **Optimization:** For local development and testing of the embedding generation, configure your container runtime to leverage local GPU hardware (e.g., passing through the RTX 4060) to avoid cloud API bottlenecks during the build phase.

Phase 2: Ingestion & Vector Pipeline (Data Engineering)
- **Deliverable:** Endpoints for document upload, parsing, and chunking.

- **Directive:** Implement an interface for the chunking logic (configurable token/character overlap). Embed the chunks and sync them to the vector store with accurate relational metadata (tags, upload date, source file). Ensure ingestion functions are idempotent.

Phase 3: RAG Core & Prompt Orchestration

- **Deliverable:** The QA endpoint.
- **Directive:** Implement the retrieval logic (top-k search). Build the context window budget mechanism—sort chunks by relevance and truncate strictly to fit the LLM's context limit. Format the prompt to enforce strict grounding and require source snippets for inline citations.

Phase 4: Frontend Integration & Telemetry (LLMOps Basics)

- **Deliverable:** Connected UI and query logging.
- **Directive:** Wire up the chat UI and admin dashboard. Implement a logging layer in FastAPI to capture query timestamps, selected collections, retrieved chunks, and the final LLM response for future LLMOps analytics.

Phase 5: CI/CD Pipeline Implementation

- **Deliverable:** Automated delivery pipeline.
- **Directive:** Establish the following stages in our CI tool:
    - **Lint/Test:** Python (ruff/pytest) and TS (eslint).
    - **Build:** Build Docker images for frontend and backend.
    - **Security Scan:** Basic vulnerability scanning on the Docker images.
    - **Push:** Push tagged images to the internal container registry.
    - **Deploy:** Trigger deployment update to the staging environment.

4. Risk Management

Risk	Mitigation Strategy
Context Window Overflow	Implement hard limits on chunk sizes and a dynamic context-trimming function before sending the payload to the LLM.
Hallucinations	System prompt must explicitly instruct the model to state "I do not know" if the answer is not present in the retrieved chunks.
API Rate Limits	Implement robust retry mechanisms with exponential backoff for all external LLM/Embedding API calls.
Environment Drift	Enforce strict use of .env files mapped into the Docker containers. No hardcoded configurations allowed.

Please review the architecture and stack decisions. If you require any clarification on the CI/CD requirements or the chunking parameters before you begin drafting the API contracts, let me know.